

DRAGO

Summer Internship Presentation

Anuja Chaudhari

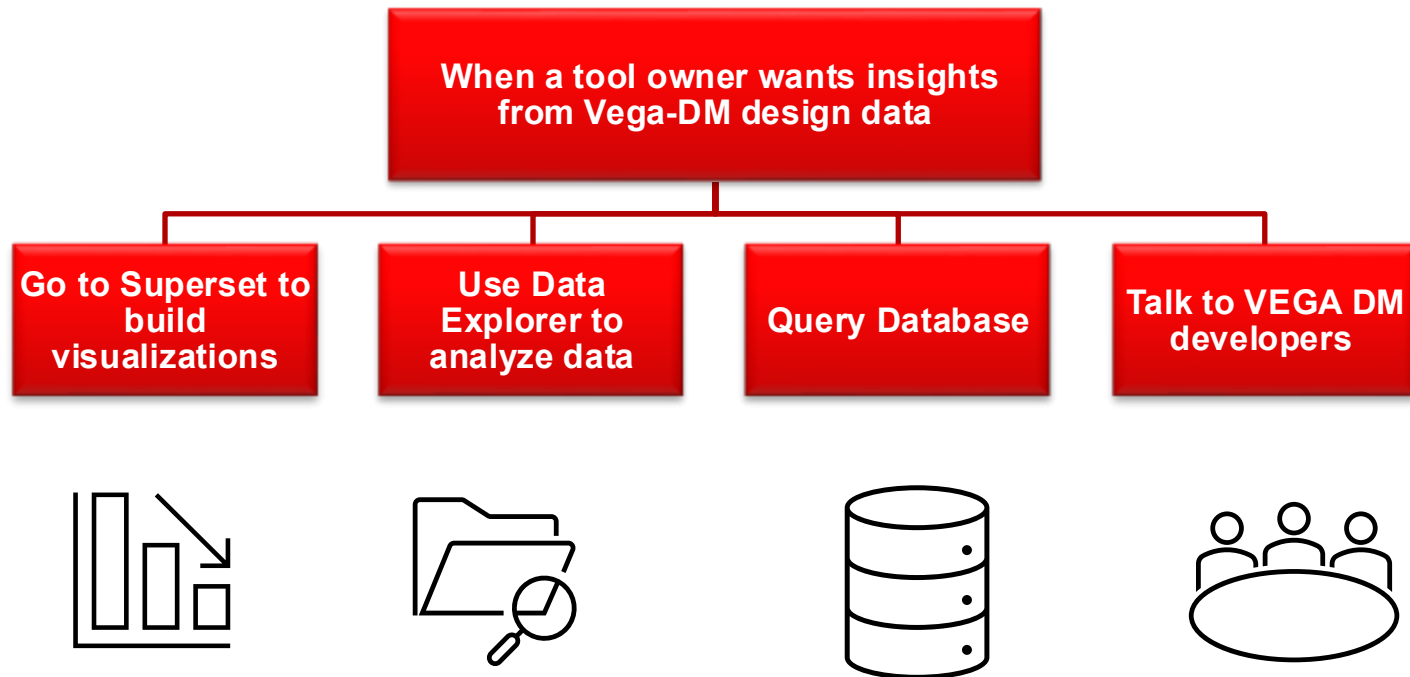
Mentor: Shreyas Jain

About me



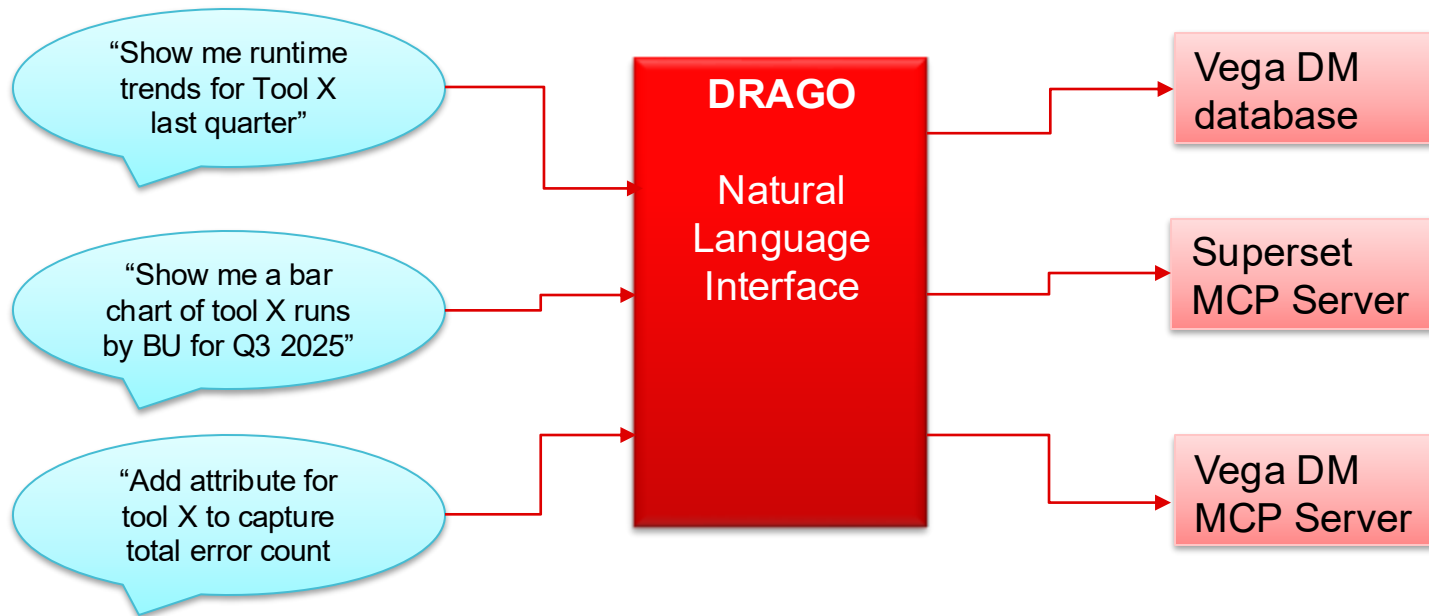
- I am Anuja, Electrical Engineering Student at IIT Gandhinagar.
- **Academic Interests:** Data Structures and Algorithms, Machine Learning
- **Beyond academics:** I play Badminton, Volleyball, Cricket. I like watching movies/series.

Problem Statement



DRAGO:

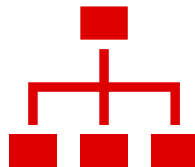
Create an AI-powered agent that understands natural language requests and translates them into actionable operations on VEGA-DM data.



Understanding the Schema



What tables
exist?

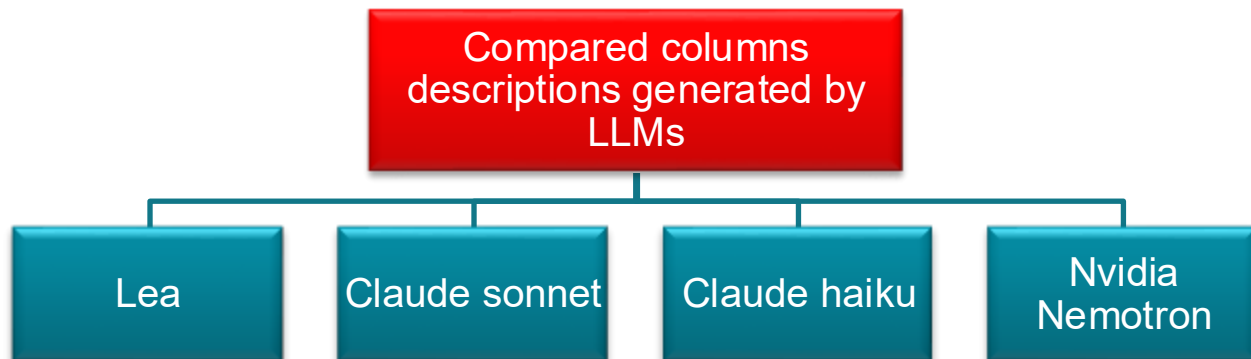


What columns
do they have?



What does each
column mean?

Generating Column Descriptions



Based on accuracy and depth of output, We decided to go ahead with Sonnet

Identifying Relevant Tables & Columns



User queries use business terms



Keyword matching breaks down when terminology doesn't align



Semantic search over these descriptions identifies the most relevant tables and columns

Ingesting Schema Into Vectara

Table name

Description:

Section 1

Column name :

Data type

Description:

Section 2

Column name:

Data type:

Description:

....(one section per column)

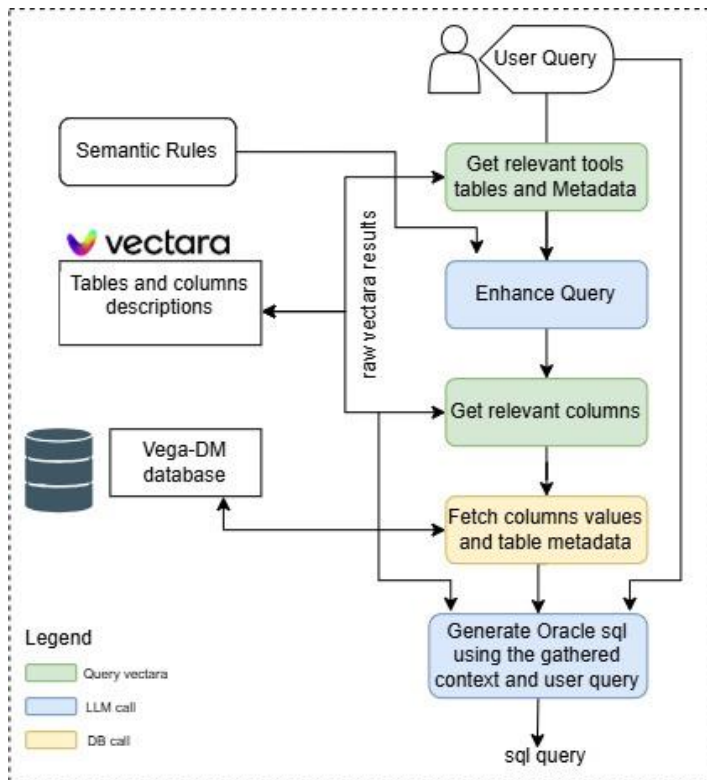
Semantic Rules

User asks: "how many runs did our BU do last quarter?"

Without semantic rules, the LLM has no way to resolve:

- What is a "run"? An invocation?
- What does "BU" map to - SBE, SBE1, or SBE2?

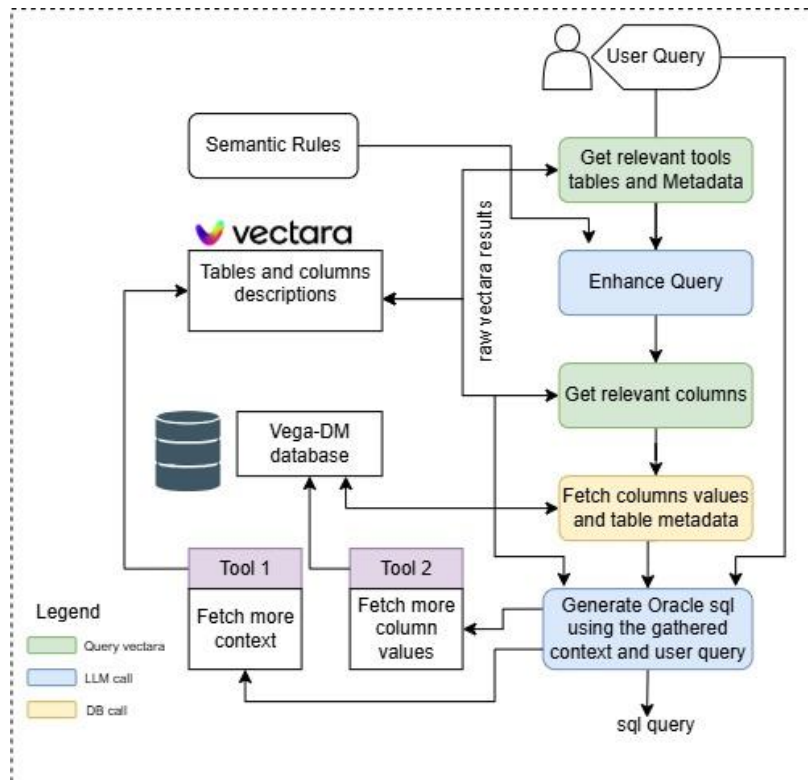
Initial Approach



Predefined Flow vs. Tool Calling

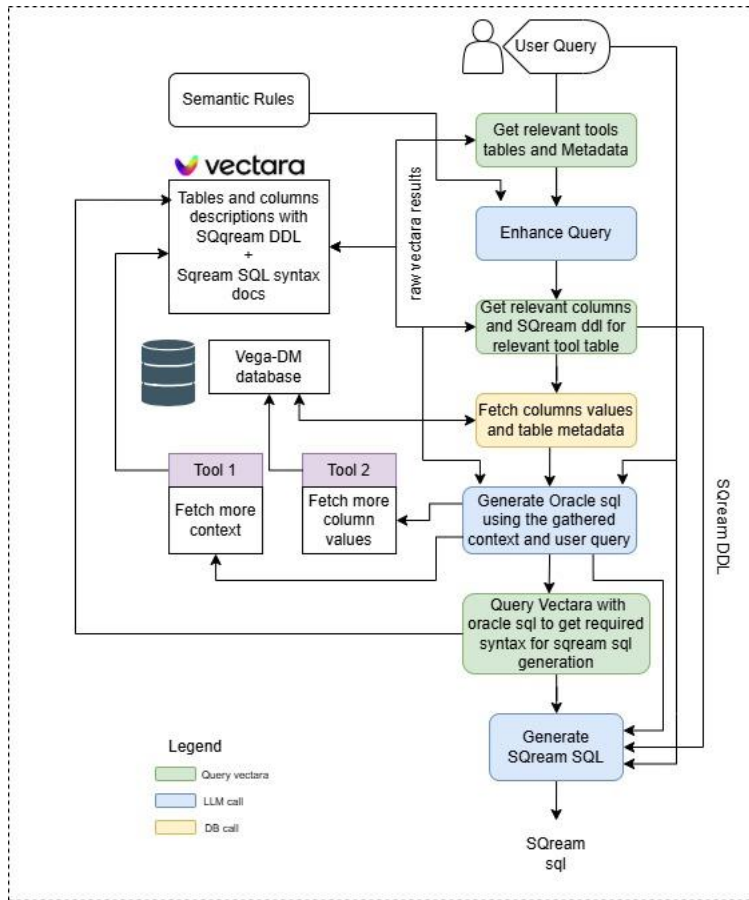
- Built an MCP server exposing tools for Vectara search, sample values retrieval, and table metadata (indexes, partitions, constraints)
- Integrated the MCP server into Claude Code to observe LLM tool-calling behavior during SQL generation
- **Finding:** LLM frequently used Vectara search and fetch sample values to gather additional context before generating SQL
- **Takeaway:** Incorporated tool-calling into the SQL generation pipeline, capped at 5 calls before forcing final output

Added Tool Calling



SQream DB Integration

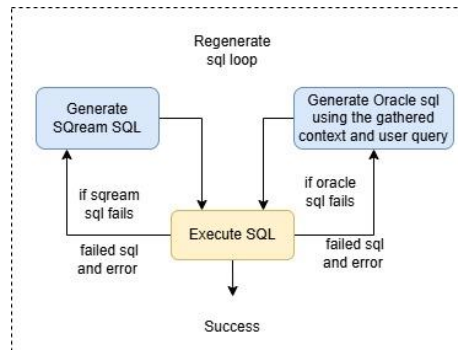
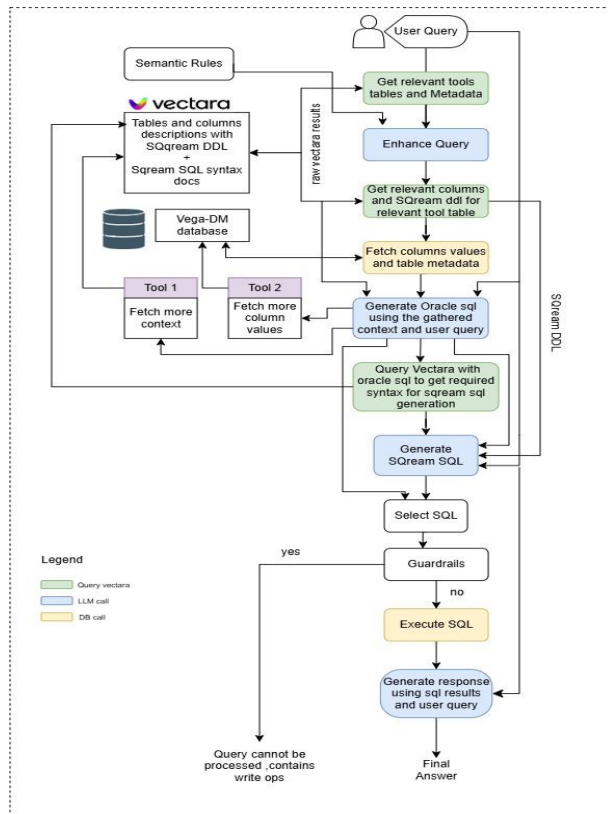
SQream DB was added as a second execution target , offering significantly faster performance on large-scale aggregations compared to Oracle.



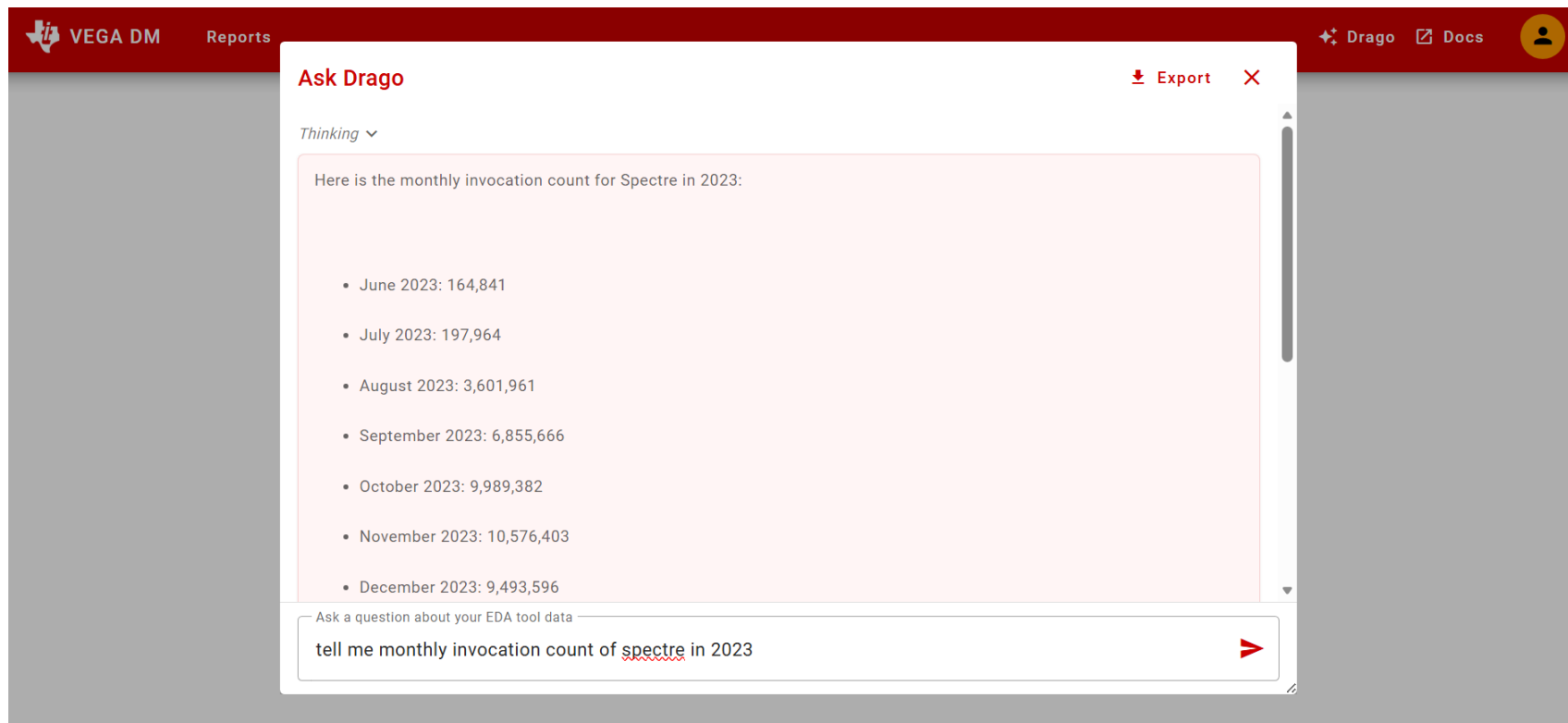
Automated Query Correction on Failure

- On execution failure, the error message and failed SQL are appended to the existing conversation history (which already contains the original query, schema context, and sample data)
- This full message history is passed back to the LLM, so it can correct the query using the same schema/context it originally used — rather than guessing blindly from the error alone
- LLM regenerates a corrected SQL query, which is retried immediately — up to a maximum of 5 attempts, falling back to a user-facing error if all attempts fail

Final Architecture



Frontend Integration



The screenshot shows the VEGA DM interface with a red header bar. The header contains the VEGA DM logo, the word 'Reports', and navigation icons for Drago, Docs, and a user profile. A chat window titled 'Ask Drago' is open, showing a 'Thinking' status and a list of monthly invocation counts for Spectre in 2023. The chat input field at the bottom contains the text 'tell me monthly invocation count of spectre in 2023'.

Ask Drago [Export] [X]

Thinking ▾

Here is the monthly invocation count for Spectre in 2023:

- June 2023: 164,841
- July 2023: 197,964
- August 2023: 3,601,961
- September 2023: 6,855,666
- October 2023: 9,989,382
- November 2023: 10,576,403
- December 2023: 9,493,596

Ask a question about your EDA tool data

tell me monthly invocation count of spectre in 2023 ➤

Frontend Integration

VEGA DM Reports

Drago Docs

Ask Drago

Export

Thinking ^

The user wants to know the monthly invocation count of spectre in 2023.

From the context:

- Spectre has TOOL_ID = 1 (mentioned in the description: "TOOL ID=1")
- The TOOL_USAGE table has INVOCATION_TIME column
- For spectre-specific query, I need to filter by TOOL_ID = 1
- User wants monthly count for year 2023
- The query should return month-wise invocation counts

Since this is a simple count operation, I don't need to join tables. I can just query TOOL_USAGE table with:

- Filter by TOOL_ID = 1 (for spectre)
- Filter by year 2023
- Group by month
- Count the invocations

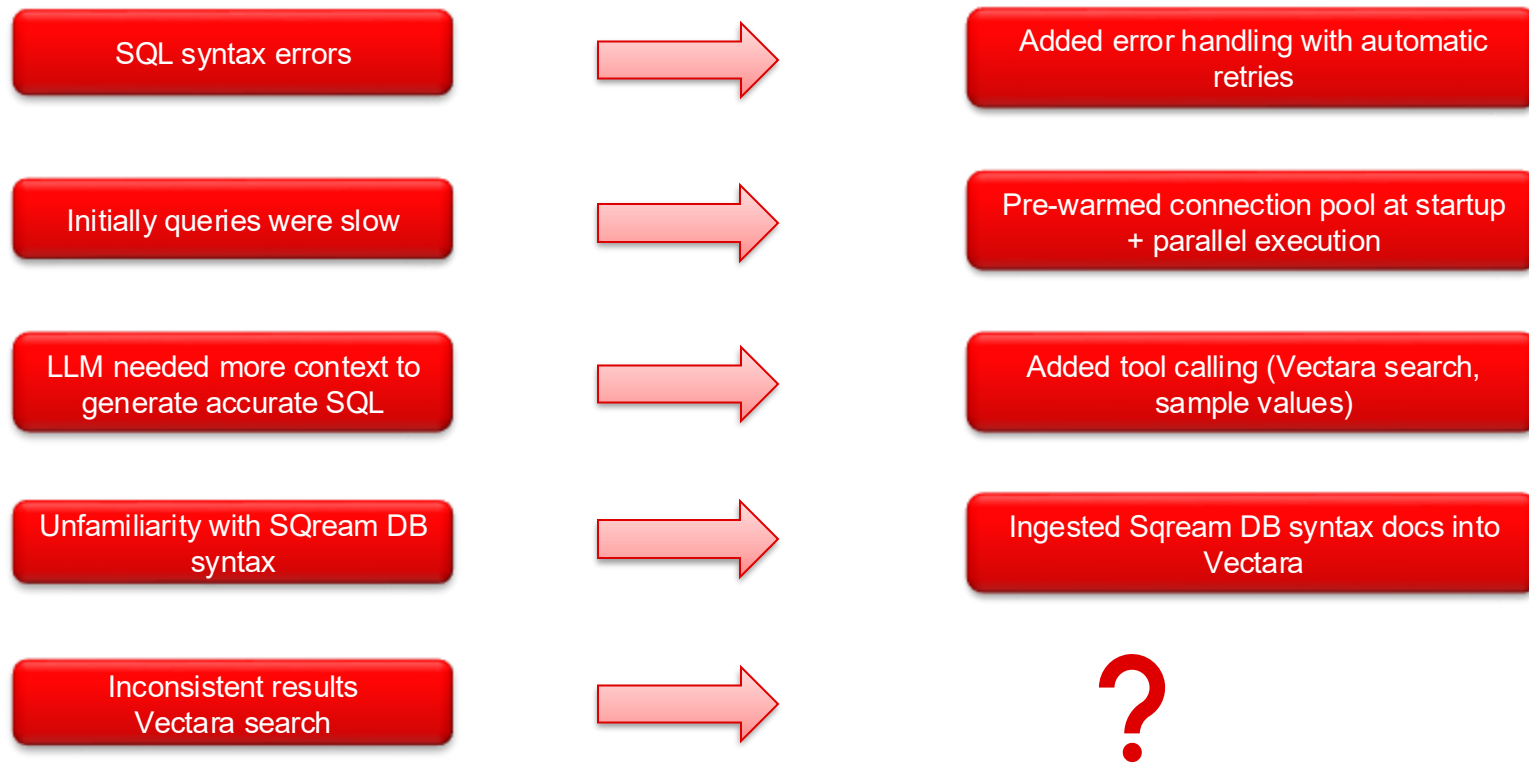
The query structure:

```
```sql
SELECT
 TO_CHAR(INVOCATION_TIME, 'YYYY-MM') as month,
 COUNT(*) as invocation_count
```

Ask a question about your EDA tool data

tell me monthly invocation count of spectre in 2023

# Development Challenges and Solutions



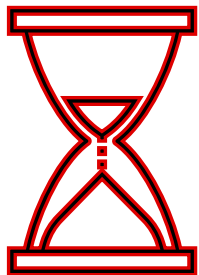
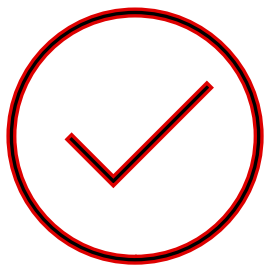
# User Feedback

Users often describe columns using business terms that don't match the actual schema

These mismatches create semantic gaps between how users speak and how the data is structured

*Feedback helps surface these semantic gaps for correction and will be used to extend the semantic rules.*

# What We're Still Optimizing



# Demo

# Future Developments



**Superset Integration:** Use Superset's REST API to generate charts directly from user queries



**Scalability to Other Schemas:** Extend the current approach inside VEGA-DM to support additional schemas which includes licensing data , galileo and onepg



**Evals:** Define evaluation benchmarks to systematically measure SQL accuracy and performance



**User Piloting:** Roll out to a limited set of users to gather real-world feedback before broader adoption

# My Learning

- Tech Stack:
  - Python
  - MCP
  - Async Programming
  - Fast API
  - Client-Server Architecture
  - SQL Alchemy
  - Oracle DB
  - Scream DB
- Writing code for production.



ORACLE

SQREAM

 FastAPI



# Thank You!



- [raaga\\_attr\\_description.csv.xlsx](#)